PyTorch Tensors: A Comprehensive Guide for Deep Learning

Educational Resource for Beginners

March 24, 2026

Contents

1 Introduction to Tensors

1.1 Definition and Fundamental Properties

A **tensor** is a specialized multidimensional array designed for mathematical and computational efficiency in deep learning frameworks. Formally, an n-dimensional tensor generalizes the concepts of scalars (0-D), vectors (1-D), and matrices (2-D) to arbitrary dimensions.

Definition 1 (Tensor)**.** *A tensor $\mathcal{T}$ of dimension n (also called rank or order) is a mathematical object that spans n directions. The shape of a tensor describes the number of elements along each dimension.*

The dimensionality of a tensor determines its geometric interpretation:

- **0-D Tensor (Scalar)**: A single numerical value with no directional extent
- **1-D Tensor (Vector)**: An array of values extending along one direction
- **2-D Tensor (Matrix)**: A grid of values extending along two orthogonal directions
- **n-D Tensor**: A generalization to n dimensions, represented as an n-dimensional array

1.2 Tensor Dimensionality and Shape

The *dimension* (or rank) of a tensor indicates how many directions it spans. Consider the following progression:

Tensor Type	Shape	Example Application
Scalar	() or [1]	Loss value in neural networks
Vector	$[n]$	Word embeddings in NLP
Matrix	$[m, n]$	Grayscale images
3-D Tensor	$[c, h, w]$	RGB images (channels × height × width)
4-D Tensor	$[b, c, h, w]$	Batches of RGB images
5-D Tensor	$[v, f, c, h, w]$	Video data (videos × frames × channels × height × width)

2 Tensor Types by Dimensionality

2.1 0-Dimensional Tensors: Scalars

A 0-dimensional tensor represents a single numerical value. In deep learning, scalars commonly appear as:

- **Loss values**: After a forward pass through a neural network, the loss function computes a single scalar value representing the prediction error:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i; \theta), y_i) \tag{1}$$

where $\mathcal{L}$ is a scalar, f is the model with parameters θ, and ℓ is a per-sample loss function.

```python
import torch

# Scalar tensor from Python value
scalar = torch.tensor(3.14)
print(scalar)           # tensor(3.1400)
print(scalar.shape)     # torch.Size([])
print(scalar.dim())     # 0
```

Listing 1: Creating scalar tensors in PyTorch

2.2 1-Dimensional Tensors: Vectors

One-dimensional tensors extend along a single direction and are mathematically equivalent to vectors. In deep learning applications:

- **Word Embeddings**: In natural language processing, each word in a vocabulary is mapped to a dense vector representation:
$$\mathbf{e}_w \in \mathbb{R}^d \tag{2}$$
where $\mathbf{e}_w$ is the embedding vector for word w and d is the embedding dimension (typically 50–300).

```
# Word embedding vector (example: 300-dimensional GloVe embedding)
embedding_dim = 300
word_vector = torch.randn(embedding_dim)  # Random initialization
print(word_vector.shape)  # torch.Size([300])
```

Listing 2: 1-D tensor representing word embeddings

2.3 2-Dimensional Tensors: Matrices

Two-dimensional tensors represent matrices and are essential for:

- **Grayscale Images**: A grayscale image of height h and width w is naturally represented as a matrix $\mathbf{I} \in \mathbb{R}^{h \times w}$, where each entry $I_{ij} \in [0, 255]$ or $[0, 1]$ represents pixel intensity.

```
# Grayscale image: 256 x 256 pixels
grayscale_image = torch.randint(0, 256, (256, 256), dtype=torch.float32)
print(grayscale_image.shape)  # torch.Size([256, 256])
```

Listing 3: 2-D tensor for grayscale images

2.4 3-Dimensional Tensors

Three-dimensional tensors add a channel dimension and are used for:

- **RGB Images**: A color image consists of three channels (Red, Green, Blue), represented as $\mathcal{I} \in \mathbb{R}^{3 \times h \times w}$. The shape convention in PyTorch is:
$$\text{shape} = [C, H, W] = [\text{channels}, \text{height}, \text{width}] \tag{3}$$

```
# RGB image: 3 channels x 256 height x 256 width
rgb_image = torch.rand(3, 256, 256)
print(rgb_image.shape)  # torch.Size([3, 256, 256])
```

Listing 4: 3-D tensor for RGB images

2.5 4-Dimensional Tensors: Batches

Deep learning models process data in *batches* for computational efficiency. A batch of B RGB images forms a 4-D tensor:

$$\mathcal{B} \in \mathbb{R}^{B \times C \times H \times W} \tag{4}$$

```
# Batch of 32 RGB images, each 128x128
batch_size = 32
batch_images = torch.rand(batch_size, 3, 128, 128)
print(batch_images.shape)  # torch.Size([32, 3, 128, 128])
```

Listing 5: 4-D tensor for batched images

2.6 5-Dimensional Tensors: Video Data

Video data extends the batch concept by adding a temporal dimension (frames). A collection of V videos, each with F frames, creates a 5-D tensor:

$$\mathcal{V} \in \mathbb{R}^{V \times F \times C \times H \times W} \tag{5}$$

```python
# 10 videos, 16 frames each, RGB, 64x64 resolution
video_batch = torch.rand(10, 16, 3, 64, 64)
print(video_batch.shape)  # torch.Size([10, 16, 3, 64, 64])
```

Listing 6: 5-D tensor for video data

3 Why Tensors Are Essential for Deep Learning

3.1 Mathematical Efficiency

Tensors enable efficient execution of fundamental linear algebra operations required by neural networks:

- **Matrix Multiplication**: The core operation in neural network layers, $\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{b}$
- **Element-wise Operations**: Activation functions applied component-wise
- **Broadcasting**: Implicit expansion of dimensions for compatible operations

3.2 Unified Data Representation

All real-world data modalities map naturally to tensors:

Data Type	Tensor Representation	Shape
Tabular data	2-D matrix	$[N, D]$ (samples × features)
Text sequences	2-D or 3-D	$[B, L]$ or $[B, L, E]$ (batch × length × embedding)
Images	4-D	$[B, C, H, W]$
Audio	2-D or 3-D	$[B, T]$ or $[B, C, T]$ (time series)
Video	5-D	$[B, T, C, H, W]$

3.3 Hardware Acceleration

Tensors are optimized for parallel computation on GPUs and TPUs. Consider the element-wise addition of two matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$:

$$\mathbf{C}_{ij} = \mathbf{A}_{ij} + \mathbf{B}_{ij}, \quad \forall i \in [1, m], j \in [1, n] \tag{6}$$

On a GPU with k cores, this operation executes in parallel across all $m \times n$ elements, achieving near-linear speedup compared to sequential CPU execution.

4 Environment Setup and Verification

Before creating tensors, verify your PyTorch installation and available hardware:

```python
import torch

# Check PyTorch version
print(f"PyTorch version: {torch.__version__}")

# Check GPU availability
if torch.cuda.is_available():
    print("GPU is available!")
    print(f"Device: {torch.cuda.get_device_name(0)}")
    device = torch.device("cuda")
else:
    print("GPU not available. Using CPU.")
```

```
    device = torch.device("cpu")
```

Listing 7: Environment verification

Tip

Always verify GPU availability before training large models. GPU acceleration can provide 10–100x speedup for tensor operations on large matrices.

5 Creating Tensors

PyTorch provides multiple methods for tensor creation, each suited to specific use cases.

5.1 Uninitialized Tensors: `torch.empty()`

The `torch.empty()` function allocates memory without initialization. The values are whatever happened to exist in that memory location.

```
# Allocate 2x3 tensor (values are arbitrary)
a = torch.empty(2, 3)
print(a)
# Example output: tensor([[4.2039e+14, 4.5640e+30, 1.0774e-38],
#                         [0.0000e+00, 0.0000e+00, 0.0000e+00]])
```

Listing 8: Creating uninitialized tensors

5.2 Zero and One Initialization

For bias initialization or creating masks:

```
# Tensor filled with zeros
zeros = torch.zeros(2, 3)
print(zeros)
# tensor([[0., 0., 0.],
#         [0., 0., 0.]])

# Tensor filled with ones
ones = torch.ones(2, 3)
print(ones)
# tensor([[1., 1., 1.],
#         [1., 1., 1.]])
```

Listing 9: Zero and one tensors

5.3 Random Initialization

Random initialization is crucial for weight matrices in neural networks:

```
# Uniform random values in [0, 1)
rand_tensor = torch.rand(2, 3)
print(rand_tensor)
# tensor([[0.1192, 0.4828, 0.9105],
#         [0.4266, 0.4203, 0.2719]])
```

Listing 10: Random tensor generation

5.3.1 Reproducibility with Random Seeds

For experimental reproducibility, set a manual seed:

```
# Set seed for reproducibility
torch.manual_seed(100)

# Generate random tensor
```

```
rand_a = torch.rand(2, 3)
print(rand_a)
# tensor([[0.1117, 0.8158, 0.2626],
#         [0.4839, 0.6765, 0.7539]])

# Reset seed and generate again
torch.manual_seed(100)
rand_b = torch.rand(2, 3)
print(rand_b)  # Identical to rand_a
```

Listing 11: Ensuring reproducibility

5.4 From Existing Data

Convert Python lists or NumPy arrays to tensors:

```
# From Python list
data = [[1, 2, 3], [4, 5, 6]]
tensor_from_list = torch.tensor(data)
print(tensor_from_list)
# tensor([[1, 2, 3],
#         [4, 5, 6]])

# From NumPy array
import numpy as np
numpy_array = np.array([[1.0, 2.0], [3.0, 4.0]])
tensor_from_numpy = torch.from_numpy(numpy_array)
```

Listing 12: Creating tensors from data

5.5 Specialized Creation Functions

```
# Range with step
arange_tensor = torch.arange(0, 10, 2)  # Start, end, step
print(arange_tensor)  # tensor([0, 2, 4, 6, 8])

# Linearly spaced values
linspace_tensor = torch.linspace(0, 10, 10)  # 10 values from 0 to 10
print(linspace_tensor)
# tensor([ 0.0000,  1.1111,  2.2222,  3.3333,  4.4444,
#          5.5556,  6.6667,  7.7778,  8.8889, 10.0000])

# Identity matrix
identity = torch.eye(5)  # 5x5 identity matrix

# Fill with specific value
full_tensor = torch.full((3, 3), 5)  # 3x3 tensor filled with 5
print(full_tensor)
# tensor([[5, 5, 5],
#         [5, 5, 5],
#         [5, 5, 5]])
```

Listing 13: Specialized tensor creation methods

6 Tensor Properties and Inspection

6.1 Shape and Dimensionality

Access tensor metadata using properties:

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])

print(x.shape)      # torch.Size([2, 3])
print(x.size())     # torch.Size([2, 3])
print(x.dim())      # 2 (number of dimensions)
print(x.numel())    # 6 (total number of elements)
```

Listing 14: Inspecting tensor properties

6.2 Creating Tensors with Matching Shapes

When you need a new tensor with the same shape as an existing one:

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])

# Same shape, uninitialized
empty_like_x = torch.empty_like(x)

# Same shape, filled with zeros
zeros_like_x = torch.zeros_like(x)

# Same shape, filled with ones
ones_like_x = torch.ones_like(x)

# Same shape, random values (must specify dtype for float)
rand_like_x = torch.rand_like(x, dtype=torch.float32)
```

Listing 15: Shape-matching tensor creation

7 Data Types and Type Conversion

7.1 Available Data Types

PyTorch supports various numeric types optimized for different use cases:

Data Type	Dtype	Description
32-bit float	`torch.float32`	Standard for deep learning
64-bit float	`torch.float64`	High precision, double memory
16-bit float	`torch.float16`	Mixed-precision training
BFloat16	`torch.bfloat16`	Reduced precision for TPUs
8-bit integer	`torch.int8`	Quantized models
32-bit integer	`torch.int32`	General indexing
64-bit integer	`torch.int64`	Large array indexing
Boolean	`torch.bool`	Masks and logical operations

7.2 Specifying and Converting Data Types

```
# Check current dtype
x = torch.tensor([1, 2, 3])
print(x.dtype)  # torch.int64 (default for integers)

# Specify dtype during creation
float_tensor = torch.tensor([1, 2, 3], dtype=torch.float32)
print(float_tensor.dtype)  # torch.float32

# Convert existing tensor
x_float = x.to(torch.float32)
# or equivalently:
x_float = x.float()

# Common conversion methods
x.double()    # Convert to float64
x.half()      # Convert to float16
x.long()      # Convert to int64
x.int()       # Convert to int32
```

Listing 16: Data type management

Important

Always ensure data type compatibility in operations. For example, `torch.rand_like()` requires floating-point output, so if the source tensor is integer type, you must explicitly specify `dtype=torch.float32`.

8 Mathematical Operations

8.1 Scalar Operations

Operations between a tensor and a single scalar value apply element-wise:

```
x = torch.rand(2, 2)

# Arithmetic with scalars
print(x + 2)    # Addition
print(x - 2)    # Subtraction
print(x * 3)    # Multiplication
print(x / 3)    # Division
print(x // 3)   # Integer division
print(x % 2)    # Modulo
print(x ** 2)   # Power
```

Listing 17: Scalar arithmetic operations

8.2 Element-wise Operations

Operations between tensors of the same shape apply element-wise:

```
a = torch.rand(2, 3)
b = torch.rand(2, 3)

# Element-wise arithmetic
print(a + b)    # Addition
print(a - b)    # Subtraction
print(a * b)    # Multiplication (Hadamard product)
print(a / b)    # Division
print(a % b)    # Modulo
print(a ** b)   # Power
```

Listing 18: Element-wise tensor operations

8.3 Mathematical Functions

```
x = torch.tensor([1.0, -2.0, 3.0, -4.0])

# Absolute value
print(torch.abs(x))      # tensor([1., 2., 3., 4.])

# Negation
print(torch.neg(x))      # tensor([-1., 2., -3., 4.])

# Rounding operations
y = torch.tensor([1.9, 2.3, 3.7, 4.4])
print(torch.round(y))    # tensor([2., 2., 4., 4.])
print(torch.ceil(y))     # tensor([2., 3., 4., 5.])
print(torch.floor(y))    # tensor([1., 2., 3., 4.])

# Clamping (constraining to range)
print(torch.clamp(y, min=2, max=3))  # tensor([2.0000, 2.3000, 3.0000, 3.0000])
```

Listing 19: Element-wise mathematical functions

8.4 Reduction Operations

Operations that aggregate values across dimensions:

```
x = torch.tensor([[1., 2., 9.],
                  [7., 6., 5.]])

# Sum all elements
print(torch.sum(x))           # tensor(30.)

# Sum along specific dimension
```

```
print(torch.sum(x, dim=0))     # Column-wise: tensor([ 8.,  8., 14.])
print(torch.sum(x, dim=1))     # Row-wise: tensor([12., 18.])

# Mean (requires float type)
print(torch.mean(x))           # tensor(5.)

# Other reductions
print(torch.median(x))         # tensor(5.)
print(torch.max(x))            # tensor(9.)
print(torch.min(x))            # tensor(1.)
print(torch.prod(x))           # Product: tensor(3780.)
print(torch.std(x))            # Standard deviation
print(torch.var(x))            # Variance

# Index of extrema
print(torch.argmax(x))         # Index of maximum: tensor(2)
print(torch.argmin(x))         # Index of minimum: tensor(0)
```

Listing 20: Reduction operations

8.5 Linear Algebra Operations

```
# Matrix multiplication
A = torch.randint(0, 10, (2, 3))
B = torch.randint(0, 10, (3, 2))
C = torch.matmul(A, B)  # or A @ B
print(C.shape)  # torch.Size([2, 2])

# Dot product
v1 = torch.tensor([1, 2])
v2 = torch.tensor([3, 4])
dot = torch.dot(v1, v2)  # 1*3 + 2*4 = 11

# Transpose
print(torch.transpose(A, 0, 1))  # or A.T, A.t()

# Determinant and inverse (square matrices only)
M = torch.tensor([[8., 2., 2.],
                  [4., 8., 8.],
                  [5., 0., 4.]])
print(torch.det(M))        # Determinant
print(torch.inverse(M))    # Matrix inverse
```

Listing 21: Matrix operations

8.6 Comparison Operations

```
x = torch.tensor([[5, 0, 1], [1, 1, 3]])
y = torch.tensor([[2, 6, 3], [5, 2, 5]])

print(x > y)    # Greater than
print(x < y)    # Less than
print(x == y)   # Equal to
print(x != y)   # Not equal to
print(x >= y)   # Greater than or equal
print(x <= y)   # Less than or equal
```

Listing 22: Comparison operations

8.7 Activation Functions

```
x = torch.tensor([[0., 6., 4.], [4., 8., 6.]])

# Log and exponential
print(torch.log(x))        # Natural logarithm
print(torch.exp(x))        # Exponential

```

```
# Square root
print(torch.sqrt(x))

# Sigmoid: $\sigma(x) = \frac{1}{1 + e^{-x}}$
print(torch.sigmoid(x))

# Softmax (along specified dimension)
print(torch.softmax(x, dim=0))

# ReLU: $\text{ReLU}(x) = \max(0, x)$
print(torch.relu(x))
```

Listing 23: Common activation functions

9 In-Place Operations

By default, tensor operations create new tensors. In-place operations modify the tensor directly, saving memory:

```
x = torch.rand(2, 3)
y = torch.rand(2, 3)

# Normal operation (creates new tensor)
z = x + y  # x remains unchanged

# In-place addition (modifies x)
x.add_(y)  # x is modified, no new tensor created

# Common in-place operations
x.relu_()      # In-place ReLU
x.mul_(2)      # In-place multiplication
x.zero_()      # Fill with zeros
x.random_()    # Fill with random values
```

Listing 24: In-place operations

Note

In-place operations are denoted by a trailing underscore (e.g., `add_`, `relu_`). Use them when memory efficiency is critical, particularly when working with large models on limited GPU memory.

10 Tensor Copying and Memory Management

10.1 The Assignment Trap

Python assignment does not create a copy; it creates a reference:

```
a = torch.rand(2, 3)
b = a  # b references the same memory as a

# Check memory location
print(id(a))  # e.g., 140234567890544
print(id(b))  # Same address!

a[0, 0] = 0   # Modify a
print(b[0, 0])  # Also 0! b was affected.
```

Listing 25: Assignment vs. copying

10.2 Proper Cloning

To create an independent copy, use `clone()`:

```
a = torch.rand(2, 3)
b = a.clone()  # Independent copy

print(id(a))  # Different from id(b)
print(id(b))

a[0, 0] = 0
print(b[0, 0])  # Unchanged!
```

Listing 26: Proper tensor cloning

11 GPU Acceleration

11.1 Device Management

Move tensors between CPU and GPU:

```
# Create device object
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Method 1: Create directly on GPU
x_gpu = torch.rand(2, 3, device=device)

# Method 2: Move existing CPU tensor to GPU
x_cpu = torch.rand(2, 3)
x_gpu = x_cpu.to(device)  # or x_cpu.cuda()

# Move back to CPU
x_cpu_again = x_gpu.cpu()

# Check device
print(x_gpu.device)  # cuda:0
```

Listing 27: Device transfer operations

11.2 Performance Comparison

The following example demonstrates GPU acceleration for large matrix multiplication:

```
import time

# Large matrices
size = 10000
A_cpu = torch.rand(size, size)
B_cpu = torch.rand(size, size)

# CPU timing
start = time.time()
C_cpu = torch.matmul(A_cpu, B_cpu)
cpu_time = time.time() - start
print(f"CPU time: {cpu_time:.4f} seconds")

# GPU timing
if torch.cuda.is_available():
    A_gpu = A_cpu.cuda()
    B_gpu = B_cpu.cuda()

    # Warm-up (GPU initialization overhead)
    torch.matmul(A_gpu, B_gpu)
    torch.cuda.synchronize()

    start = time.time()
    C_gpu = torch.matmul(A_gpu, B_gpu)
    torch.cuda.synchronize()  # Wait for GPU to finish
    gpu_time = time.time() - start

    print(f"GPU time: {gpu_time:.4f} seconds")
    print(f"Speedup: {cpu_time/gpu_time:.1f}x")
```

Listing 28: CPU vs. GPU performance benchmark

Tip

Always include `torch.cuda.synchronize()` when timing GPU operations, as CUDA operations are asynchronous by default.

12 Tensor Reshaping

12.1 Basic Reshaping

```
x = torch.arange(16)  # [0, 1, 2, ..., 15]
print(x.shape)  # torch.Size([16])

# Reshape to 4x4
x_4x4 = x.reshape(4, 4)  # or x.view(4, 4)

# Flatten to 1-D
x_flat = x_4x4.flatten()  # or x_4x4.view(-1)

# Remove dimensions of size 1
y = torch.rand(1, 3, 1, 4)
y_squeezed = y.squeeze()  # Shape: [3, 4]

# Add dimension of size 1
y_unsqueezed = y_squeezed.unsqueeze(0)  # Shape: [1, 3, 4]
```

Listing 29: Reshaping operations

12.2 Permutation of Dimensions

```
# Image batch: [Batch, Channels, Height, Width] = [2, 3, 4]
x = torch.rand(2, 3, 4)
print(f"Original shape: {x.shape}")  # [2, 3, 4]

# Permute to [Channels, Batch, Width, Height] = [3, 2, 4]
x_perm = x.permute(1, 0, 2)
print(f"Permuted shape: {x_perm.shape}")  # [3, 2, 4]
```

Listing 30: Dimension permutation

12.3 Practical Example: Batch Preparation

```
# Single image: [C, H, W] = [3, 226, 226]
single_image = torch.rand(3, 226, 226)

# Add batch dimension for model input: [1, C, H, W]
batched_image = single_image.unsqueeze(0)
print(batched_image.shape)  # torch.Size([1, 3, 226, 226])

# Remove batch dimension after inference
output = batched_image.squeeze(0)
print(output.shape)  # torch.Size([3, 226, 226])
```

Listing 31: Adding batch dimension for inference

13 Interoperability with NumPy

PyTorch tensors and NumPy arrays share memory when possible, enabling zero-copy interoperability:

```python
import numpy as np
import torch

# PyTorch tensor to NumPy array
torch_tensor = torch.rand(3, 3)
numpy_array = torch_tensor.numpy()  # Shares memory if on CPU

# NumPy array to PyTorch tensor
numpy_array = np.array([[1., 2.], [3., 4.]])
torch_tensor = torch.from_numpy(numpy_array)  # Shares memory

# Explicit copy if needed
torch_copy = torch.tensor(numpy_array)  # Always copies
```

Listing 32: PyTorch-NumPy conversion

Important

The `numpy()` method only works for tensors on CPU. GPU tensors must be moved to CPU first using `.cpu()` before conversion to NumPy.

14 Conclusion

This document has presented a comprehensive foundation for working with PyTorch tensors. Key takeaways include:

1. **Tensors generalize scalars, vectors, and matrices** to arbitrary dimensions, enabling unified representation of all data types in deep learning.
2. **Dimensionality matters**: Understanding shapes like $[B, C, H, W]$ for images or $[V, F, C, H, W]$ for video is essential for model architecture design.
3. **Hardware acceleration**: GPU utilization through `.cuda()` or `.to(device)` provides order-of-magnitude speedups for large-scale computations.
4. **Memory efficiency**: In-place operations (`add_`, `relu_`) and proper cloning (`clone()`) are critical for training large models.
5. **Type safety**: Attention to `dtype` specifications prevents runtime errors, particularly with functions requiring specific numeric types.

With these fundamentals, you are prepared to implement neural network architectures, custom loss functions, and data preprocessing pipelines using PyTorch's tensor operations.